

// CYBR.SEC.CON 2026

I'm a Hacker, Not a Developer

What I Learned Building AI Tools



“I really wish there was a way to...”

– YOU, PROBABLY



// ABOUT ME

Jason Mull

Senior Cybersecurity Architect · Lockstep Technology Group



FOCUS

Offensive Security & Incident Response



BACKGROUND

Consulting & MSP — systems and network engineering



EDUCATION

Louisiana State University & SANS
Technology Institute



BASED IN

Baton Rouge, Louisiana

What We'll Cover

01 Prompt Harder

Writing code is easy now. The challenge is describing what you want.

02 Give it Reach

How LLMs can interact with the tools you already use every day.

03 Keeping Your Data In-House

Local models can be a rabbit hole. Here's what to think through.

04 Audit the Machine

I said writing code was easy. That doesn't always mean the code is good.

// LESSON 01

Prompt Harder

// PROMPT HARDER

We've Been Through This Before

Hand-written HTML, custom PHP

→

WordPress

CPAs for personal taxes,
Understanding the tax code

→

TurboTax, H&R Block

Writing code

→

**Using Subject Matter Expertise and
describing what you want**

The specialized thing becomes common. Again.

// PROMPT HARDER

Prompting Better

1 **Just Ask** — Rough description, no structure.

Fits a narrow ask, where a wrong answer is obvious.

2 **Write the Spec** — Structured prompt by hand. Consistent, but daunting.^[03]

Fits work where you already know the requirements.

3 **Get Interviewed** — Provide the basic, high-level goal. The model asks questions and structures the answer.^[19]

Fits ambiguous scope — questions surface decisions.



Andrej Karpathy
@karpathy

One pattern I find useful for working with LLMs is a nice long ramble session. Sometimes the LLM needs more bits to understand what you're trying to achieve, but you're too lazy to type them. In these cases I like to lean back, switch to /voice and just ramble for like 10 minutes, total mess, anything goes, full stream of consciousness...

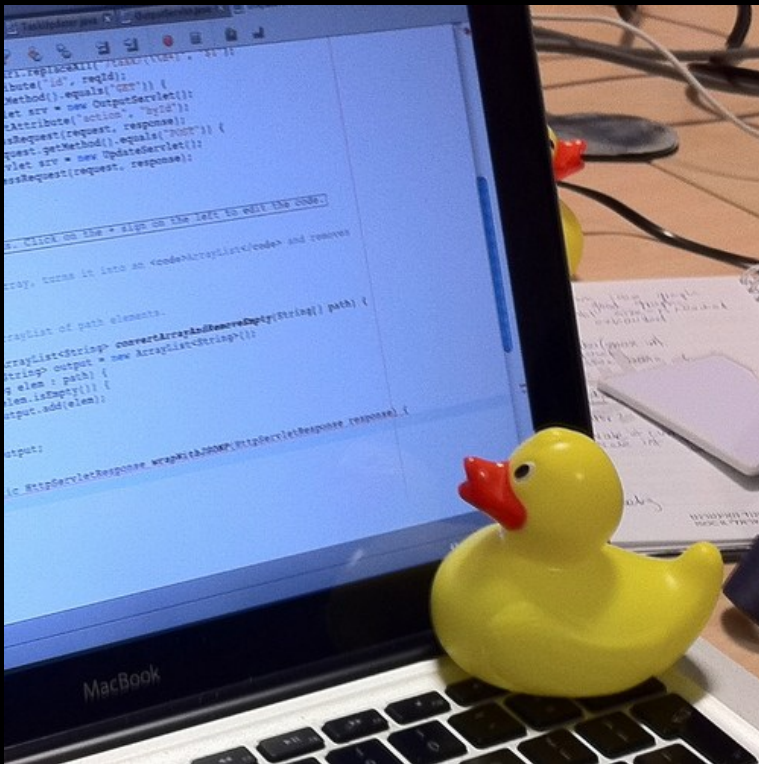
via X · July 21, 2026 (excerpt)



x.com/karpathy · 21 Jul 2026 [19]

// PROMPT HARDER

Rubber Duck Debugging... Evolved.



Talking it out isn't new.

Rubber duck debugging has worked for decades. Explaining the problem out loud forces you to find the gap yourself. ^[20]

AI takes it one step further.

The duck starts asking questions back, interrogating your answers for more clarity.

Present it high-level, refine through Q&A.

Start with the problem in ordinary language. Let the model ask about missing requirements, then review a concise specification before implementation. Use more discussion where the work is unclear.

// PROMPT HARDER

What Makes a Specification?

Purpose

One paragraph, scope not features

Constraints

Environment, platforms, what it runs against

Anti-goals

The things you DON'T want the tool to do

Acceptance criteria

What evidence it must show, and how it fails

Design decisions

Choices, and why

Domain requirements

The rules your expertise supplies

Session startup

What to re-read to be useful again

Where it lives

Repo root: CLAUDE.md, AGENTS.md, your tool's file

// LESSON 02

Give it Reach

// GIVE IT REACH

Model Context Protocol (MCP)

IT IS

A protocol^[01]

A way to expose capability

Plumbing

IT ISN'T

A model

A framework you have to adopt

A replacement for your API

// GIVE IT REACH

Integration Targets

Same benefit every time: LLM reach into tools you already own, scoped down to the specific capabilities you want

SERVICE TYPE

EXAMPLES

Hosted services with APIs

SaaS platforms, vendor consoles, internal services

Local applications and CLI tools

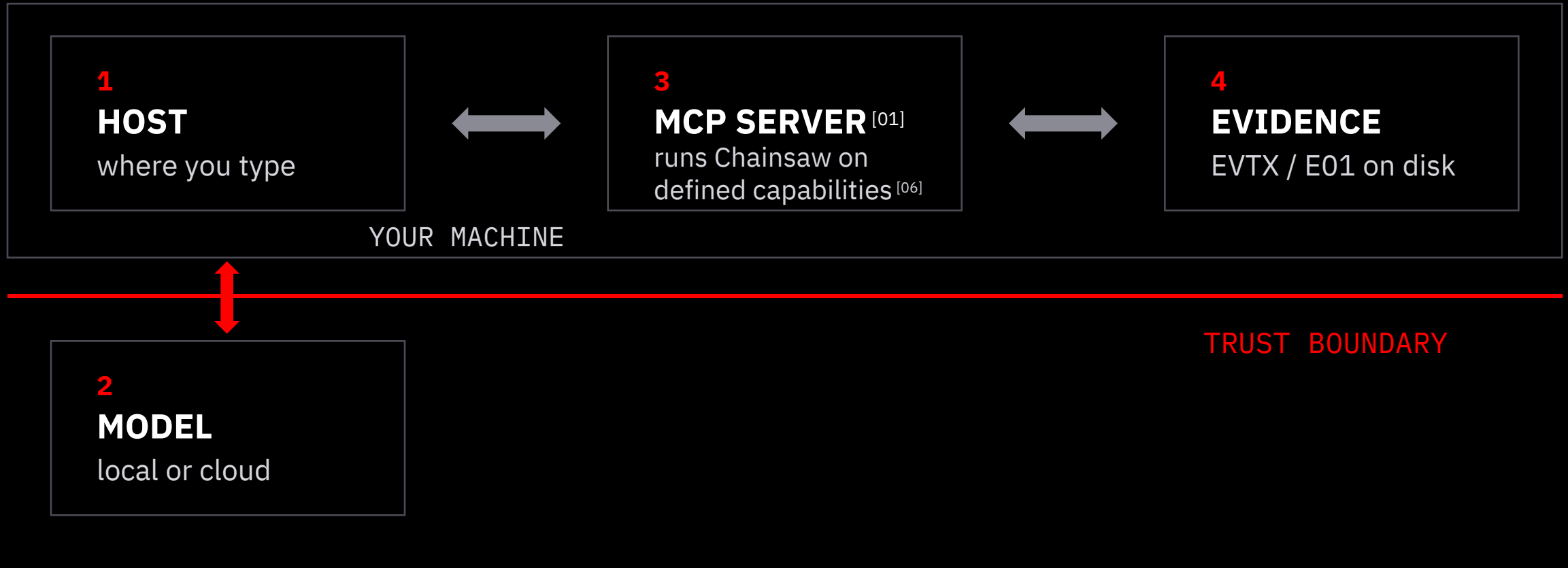
Chainsaw, nmap, git, anything with a CLI

Local data stores

Filesystems, databases, log stores, evidence directories

// GIVE IT REACH

Host, Server, Tools



// GIVE IT REACH

Finding the Use Case

01 Do you run it by hand, often?

Some of the best projects are those that involve repeated tasks

02 Does it take structured input and produce structured output?

Structured data makes it easier for the LLM to read input and provide clearer output

03 Is the expensive part the analysis, not the running?

Fixed sequence, use a script. Branching decisions, use an agent.

04 Can I tell when it worked, and limit the damage when it doesn't?

Start with a bounded task and reviewable output — not open access to a live case.

// LESSON 03

Keeping Your Data In-House

 **Hugging Face**

Models

Datasets

Spaces

Buckets NEW

Docs

Enterprise

Pricing

Main

Tasks

Libraries

Languages

Licenses

Other

Tasks

Text Generation

Any-to-Any

Image-Text-to-Text

Image-to-Text

Image-to-Image

Text-to-Image

Text-to-Video

Text-to-Speech

+ 44

Parameters

< 1B

6B

12B

32B

128B

> 500B

Libraries

PyTorch

TensorFlow

JAX

Transformers

Diffusers

GGUF

MLX

Transformers.js

Safetensors

+ 44

Apps

vLLM

llama.cpp

MLX LM

LM Studio

Ollama

Jan

Draw Things

+ 10

Inference Providers

Groq

Novita

Cerebras

Nscale

fal

Together AI

Fireworks

Featherless AI

+ 10

Hardware

Models

2,980,725

Filter by name

Base only

Inference Available

Sort: Most likes

black-forest-labs/FLUX.1-dev

Text-to-Image • 12B • Updated Jun 27, 2025 • 481k • 14.1k

moonshotai/Kimi-K3

Image-Text-to-Text • 2.8T • Updated 14 days ago • 1.51M • 10.5k

CompVis/stable-diffusion-v1-4

Text-to-Image • 0.9B • Updated Aug 23, 2023 • 412k • 7.05k

meta-llama/Meta-Llama-3-8B

Text Generation • 8B • Updated Sep 27, 2024 • 1.91M • 6.63k

openai/whisper-large-v3

Automatic Speech Recognition • 2B • Updated Au... • 4.9M • 6.13k

deepseek-ai/DeepSeek-V4-Pro

Text Generation • 1.6T • Updated Jun 22 • 1.44M • 5.4k

Tongyi-MAI/Z-Image-Turbo

Text-to-Image • 6B • Updated Jan 30 • 998k • 5.1k

bigscience/bloom

Text Generation • 176B • Updated 13 days ago • 12.4k • 5.03k

zai-org/GLM-5.2

Text Generation • 753B • Updated Jul 2 • 2.5M • 4.92k

deepseek-ai/DeepSeek-R1

Text Generation • 685B • Updated Mar 26, 2025 • 8.69M • 13.6k

stabilityai/stable-diffusion-xl-base-1.0

Text-to-Image • 3B • Updated Oct 30, 2023 • 1.38M • 8.03k

hexgrad/Kokoro-82M

Text-to-Speech • Updated Apr 10, 2025 • 11.5M • 6.67k

meta-llama/Llama-3.1-8B-Instruct

Text Generation • 8B • Updated Sep 25, 2024 • 7.37M • 6.57k

black-forest-labs/FLUX.1-schnell

Text-to-Image • 12B • Updated Aug 16, 2024 • 199k • 5.51k

sentence-transformers/all-MiniLM-L6-v2

Sentence Similarity • 22.7M • Updated Jun 1 • 241M • 5.2k

openai/gpt-oss-120b

Text Generation • 117B • Updated Aug 26, 2025 • 4.01M • 5.09k

stabilityai/stable-diffusion-3-medium

Text-to-Image • Updated Aug 12, 2024 • 8.27k • 5.01k

openai/gpt-oss-20b

Text Generation • 22B • Updated Aug 26, 2025 • 7.91M • 4.9k

// KEEPING YOUR DATA IN-HOUSE

Why Run an LLM Yourself?

Privacy is the practical one.

Incident response data comes with lawyers, insurers, and NDAs attached. Provider agreements might cover me. Not sending it at all is better.

No refusals is the other one.

Hosted models refuse some legitimate security work — phishing pretexts, exploit code, anything that reads as offensive out of context. You can run local models to bypass these restrictions.

LIMITS

No rate caps, no context window someone else set.

CURIOSITY

Sometimes you just want to see how it works.

// KEEPING YOUR DATA IN-HOUSE

Getting Started

-
- | | | |
|-----------|--|--|
| 01 | LM Studio ^[07]
GUI. Browse, download, one-click local server. | Start here. You'll be running something in twenty minutes. |
|-----------|--|--|
-
- | | | |
|-----------|---|---|
| 02 | Ollama ^[08]
CLI. <code>ollama run <model></code> | CLI. Can run in Docker. Straightforward to get up and running. Lacks performance customization options. |
|-----------|---|---|
-
- | | | |
|-----------|--|--|
| 03 | llama.cpp ^[09]
The engine underneath most of the above. | When you need real control over context and offload. |
|-----------|--|--|
-

// KEEPING YOUR DATA IN-HOUSE

Picking a Model

`fdtn-ai/Foundation-Sec-8B-Reasoning-Q4_K_M-GGUF`

PUBLISHER

Who published this file. Often a repack — check who actually trained it.

FAMILY

Foundation-Sec, security-tuned by Cisco. A candidate to evaluate, not a guarantee.^[10]

SIZE

Parameter count, roughly capacity. 30B-A3B: 30B of memory, ~3B active per token. Not dense-3B fast.

REASONING

Base — not tuned for instructions. Instruct — your default. Reasoning — steps; slower, wordier.

QUANT

How much precision was traded away to shrink it. Q4 is the usual sweet spot.

Model Decision Path

01 Capability

Tool calling, image/audio generation, structured output, context length. Binary — check this first.

02 Fit

Memory has to cover weights, context cache and runtime overhead. Start with VRAM. With MoE, most of the model's weights sit in system RAM, and only the experts a given token actually needs get pulled into VRAM — so a bigger MoE model can run on a smaller GPU than a dense model that size. Leave 1–2GB for the OS.

03 Suitability

Trained on the right data for your domain? Security-tuned is a candidate, not proof. Test it.

// KEEPING YOUR DATA IN-HOUSE

Fitting Isn't the Same as Performing

Tokens/sec	Below ~10 tok/s starts to feel slow to read along with.
Time to first token	Grows with context length. Worst with a full context window.
Tool calling	Give it a real tool schema and see if it calls it.
Accuracy	Feed it data you already know the answer to. Does it get it right?

llmfit estimates before you download. AIPerf measures the truth once it's live. ^[11] ^[12]

Search

Press / to search...

Providers (P)

All

Use Case (U)

All

Caps (C)

All

Sort [s]

Date

Fit [f]

Perfect

Avail [a]

All

TP [T]

All

Theme [t]

Default

Models (779/1085)

Inst	Model	Provider	Params	Score	tok/s*	Quant	Disk	Mode	Mem %	Ctx	Date ▼	Fit	Use Case
●	deepseek-ai/DeepSeek-Coder-V2-Lite-Instruct	DeepSeek	15.7B	95	137	Q6_K	12.6G	GPU	50%	163k	2024-06	Perfect	Coding
●	RedHatAI/DeepSeek-Code	redhat	15.7B	95	150	Q6_K	12.6G	GPU	50%	163k	—	Perfect	Coding
●	deepseek-ai/DeepSeek-V2-Chat	DeepSeek	15.7B	94	150	Q6_K	12.6G	GPU	50%	163k	—	Perfect	Chat
●	NVFP4/Qwen3-30B-A3B-Instruct	nvfp4	15.6B	93	193	Q6_K	12.5G	GPU	50%	262k	—	Perfect	Chat
●	cyankiwi/Qwen3-Coder-72B	cyankiwi	14.4B	93	498	AWQ-4bit	7.2G	GPU	46%	262k	—	Perfect	Coding
●	bullpoint/Qwen3-Coder-72B	bullpoint	14.4B	93	498	AWQ-4bit	7.2G	GPU	46%	262k	—	Perfect	Coding
●	deepseek-ai/DeepSeek-V2-Chat	DeepSeek	15.7B	93	150	Q6_K	12.6G	GPU	50%	163k	—	Perfect	General
●	nvidia/Qwen3-30B-A3B-Instruct	nvidia	15.6B	92	193	Q6_K	12.5G	GPU	50%	40k	—	Perfect	General
●	moonshotai/Moonlight-1	moonshotai	16.0B	92	285	Q6_K	12.8G	GPU	51%	8k	—	Perfect	Chat
●	Qwen/Qwen1.5-MoE-A2.7B	Alibaba	14.3B	92	202	Q6_K	11.5G	GPU	46%	8k	—	Perfect	General
●	casperhansen/mistral-nemo	casperhansen	12.2B	92	40.2	AWQ-4bit	6.1G	GPU	47%	1024k	—	Perfect	Chat
●	moonshotai/Moonlight-1	moonshotai	16.0B	92	285	Q6_K	12.8G	GPU	51%	8k	—	Perfect	General
●	inclusionAI/LLaDA2.0-m	inclusionai	16.3B	92	254	Q6_K	13.0G	GPU	52%	32k	—	Perfect	General
●	inclusionAI/LLaDA2.1-m	inclusionai	16.3B	92	254	Q6_K	13.0G	GPU	52%	32k	—	Perfect	General
●	inclusionAI/Ling-Lite	inclusionai	16.8B	92	141	Q6_K	13.4G	GPU	54%	32k	2025-02	Perfect	General
●	ai-sage/GigaChat3-10B	ai-sage	11.5B	91	331	Q8_0	12.1G	GPU	37%	262k	—	Perfect	Chat
●	cyankiwi/Qwen3-Next-80B	cyankiwi	14.7B	91	488	AWQ-4bit	7.4G	GPU	47%	262k	—	Perfect	General
●	Qwen/Qwen2.5-Coder-14B	Alibaba	14.8B	90	33.4	AWQ-4bit	7.4G	GPU	56%	32k	—	Perfect	Coding
●	cyankiwi/InternVL3_5-3	cyankiwi	12.1B	90	40.9	AWQ-4bit	6.0G	GPU	43%	4k	—	Perfect	General
●	solidrust/gemma-2-9b-it	solidrust	10.2B	90	48.5	AWQ-4bit	5.1G	GPU	39%	8k	—	Perfect	General
●	dengcao/GLM-4.1V-9B-Thinking	dengcao	10.4B	89	47.6	AWQ-4bit	5.2G	GPU	40%	65k	—	Perfect	General
●	unsloth/Qwen3-Coder-NeMo	unsloth	Unknown	89	35.2	Q8_0	7.4G	GPU	52%	32k	2026-02	Perfect	Coding
●	bigcode/starcoder2-7b	BigCode	7.2B	88	34.3	Q8_0	7.5G	GPU	53%	16k	2024-02	Perfect	Coding
●	InternScience/Agents-A	InternScienc	Unknown	88	35.2	Q8_0	7.4G	GPU	51%	4k	2026-07	Perfect	Multimodal
●	deepseek-ai/DeepSeek-R1	deepseek-ai	8B	88	30.8	Q8_0	8.4G	GPU	62%	8k	2025-01	Perfect	Reasoning
●	Qwen/Qwen2.5-Coder-7B	Alibaba	7.6B	88	32.4	Q8_0	8.0G	GPU	56%	32k	2024-09	Perfect	Coding
●	Qwen/Qwen2.5-Coder-7B	Alibaba	7.6B	88	32.4	Q8_0	8.0G	GPU	56%	32k	—	Perfect	Coding
●	Qwen/Qwen2.5-Coder-7B	Alibaba	7.6B	88	64.7	GPTQ-Int4	3.8G	GPU	30%	32k	—	Perfect	Coding
●	Qwen/Qwen2.5-Coder-7B	Alibaba	7.6B	88	64.7	AWQ-4bit	3.8G	GPU	30%	32k	—	Perfect	Coding
●	Qwen/Qwen2-VL-7B-Instr	Alibaba	8.3B	88	59.4	AWQ-4bit	4.1G	GPU	33%	32k	—	Perfect	Chat
●	Qwen/Qwen2-VL-7B-Instr	Alibaba	8.3B	88	59.4	GPTQ-Int4	4.1G	GPU	33%	32k	—	Perfect	Chat
●	Qwen/Qwen2.5-VL-7B-Ins	Alibaba	8.3B	88	59.4	AWQ-4bit	4.1G	GPU	33%	128k	—	Perfect	Chat
●	deepseek-ai/DeepSeek-R1	DeepSeek	7.6B	87	32.4	Q8_0	8.0G	GPU	56%	131k	2025-01	Perfect	Reasoning
●	RedHatAI/Meta-Llama-3.1	redhat	8.0B	87	61.4	GPTQ-Int4	4.0G	GPU	32%	131k	—	Perfect	Chat
●	deepseek-ai/DeepSeek-R1	DeepSeek	8.2B	87	30.1	Q8_0	8.6G	GPU	61%	131k	—	Perfect	Reasoning
●	Momix-44/Qwen3.5-9B-ge	momix-44	9.4B	87	26.2	Q8_0	9.9G	GPU	69%	262k	—	Perfect	Reasoning

NVIDIA AIPerf | LLM Metrics

Metric	avg	min	max	p99	p90	p50	std
Time to First Token (ms)	216.07	115.17	248.73	246.85	238.47	234.16	42.52
Time to Second Token (ms)	12.83	12.72	12.93	12.93	12.91	12.84	0.06
Time to First Output Token (ms)	1,129.62	220.97	2,559.25	2,512.09	2,087.61	1,025.26	931.36
Request Latency (ms)	3,475.62	3,368.37	3,504.50	3,503.78	3,499.25	3,491.19	43.22
Inter Token Latency (ms)	12.84	12.80	12.89	12.89	12.88	12.84	0.03
Output Token Throughput Per User (tokens/sec/user)	77.86	77.56	78.12	78.11	78.04	77.87	0.16
E2E Output Token Throughput (tokens/sec/user)	73.32	72.60	75.70	75.69	75.48	72.95	0.96
Output Sequence Length (tokens)	254.80	254.00	255.00	255.00	255.00	255.00	0.40
Input Sequence Length (tokens)	512.00	512.00	512.00	512.00	512.00	512.00	0.00
Output Token Throughput (tokens/sec)	73.23	N/A	N/A	N/A	N/A	N/A	N/A
Request Throughput (requests/sec)	0.29	N/A	N/A	N/A	N/A	N/A	N/A
Request Count (requests)	20.00	N/A	N/A	N/A	N/A	N/A	N/A

FoundationSec doesn't support tool calling.

Tool calling is not a feature of an MCP server. It is the premise.

// KEEPING YOUR DATA IN-HOUSE

Putting a Model Into Action

```
llama-server --host 0.0.0.0
--port 8080 -hf fdtn-
ai/Foundation-Sec-8B-
Reasoning-Q4_K_M-
GGUF:Q4_K_M -ngl 99 -c
32768 -fa on --jinja --temp
0.7 --top-p 0.95 --cache-
type-k q8_0 --cache-type-v
q8_0 --parallel 1
```

- | | |
|----------|--|
| -hf | Downloads and caches the model for you. No manual file wrangling. |
| --jinja | Formats prompts as this model expects. Without it, tool calls break. |
| -ngl 99 | Runs the model entirely on GPU. Too low and generation slows badly. |
| -c 32768 | How much conversation it holds. Too big for VRAM, it won't start. |

// LESSON 04

Audit the Machine

// AUDIT THE MACHINE

How to Not Vibe Code

01 AI generates the code

— ~~Review it~~

02 I test it

03 If it works, I move on

04 If it doesn't, I have another conversation with the model

// AUDIT THE MACHINE

Build This Tool. Make No Mistakes.

Same code. Different presentation. Completely different scrutiny.

AI output arrives formatted, commented, confident. A human's pull request arrives looking like a draft, so it gets treated like one. As a result, we might have a tendency to approve the AI-generated code without review.

Reviewing code you can't write means you're proofreading a language you don't speak.

When it comes to non-developers making their own software, do they have the experience and knowledge to properly understand and audit their own projects?

// AUDIT THE MACHINE

Build This Tool. Make No Mistakes.

Learn what command injection looks like — that's why we're all in this room. Until you can spot it cold, run something that points at the line.

```
/plugin install claude-  
security@claude-plugins-official  
/claude-security
```

Anthropic's own. No API key, no CI.^[15]

Settings > Code security > CodeQL

Free on public repos. Scans pushes and PRs to your default branch.^[16]

HIGH Unauthenticated HTTP transport with wildcard CORS

The streamable-HTTP MCP endpoint ships with no authentication and `allow_origins=["*"]`. Any origin that can reach the port can drive the full tool set — including a browser tab open on an unrelated site reaching the default loopback binding — and read case data back.

```
src/chainsawmcp/server.py : 788-819
```

MEDIUM No case-scoping on evidence paths

`prepare_evidence` / `start_hunt` accept any filesystem path with no allowlist against the configured case directory. The OpenWebUI `!analyse <path>` trigger exposes this to any chat user with no per-user check, letting one analyst pull another case's evidence into their own session.

```
server.py (prepare_evidence, start_hunt) + extras/chainsaw_filter.py
```

// AUDIT THE MACHINE

AI Can Invent Dependencies

Slopsquatting: ask for code, get a package name that sounds right. Sometimes it doesn't exist — and whoever registers that name owns everyone who installs it.

19.7%

of AI-suggested packages
were hallucinated

Spracklen et al., USENIX Security 2025 ^[17]

43%

come back with the same
name every run — so they're
predictable

237

repos copied **react-**
codeshift into their
setup files

aikido.dev, Jan 2026 ^[18]

// AUDIT THE MACHINE

Check Before You Install

-
- 01 Does it exist? No registry page, no package. pypi.org · npmjs.com
 - 02 Is it new? A real library has years of downloads. A potential trap might have days. Check the date it was first published, not just that a page exists.
 - 03 Real project behind it? Follow the Source link — commits, issues, real people.
 - 04 Do you know why it's there? If you can't say what it does, ask first.
-

Local Isn't the Same as Safe

- 01 **Verify what it's bound to.** Default is loopback, but `OLLAMA_HOST=0.0.0.0` is in half the tutorials and binds all IPv4. **Do:** validate listening ports and firewall as needed. Ideally, use Tailscale, a Cloudflare Tunnel plus an Access policy, or similar for external use.
- 02 **Understand the model format you use.** `.pt/.pth` are python pickle formats — loading one can run arbitrary code. Safetensors and GGUF's tensor data doesn't — but GGUF's chat template is live templating logic that runs on every prompt and can quietly alter what the model sees or does. **Do:** prefer safetensors/GGUF, pin publisher and digest, and read the chat template before you trust it. ^[22]^[21]
- 03 **Your inference server is an application.** CVE-2026-7482 was a parser bug in Ollama's GGUF loader, not a pickle one. **Do:** fixed in 0.17.1 — check `ollama -v` against releases on a schedule you own. ^[08]^[14]
- 04 **Generated code still needs review.** USENIX found at least 5% hallucination for GPT-series vs 21.7% for open-source on 2023–24 models; the spread has narrowed since. **Do:** pin versions, run pip-audit, read the diff. Judge models on measured results, not where they run. ^[17]

Closing Remarks

// LET'S TALK

Thank you.

PRESENTATION

<https://bit.ly/NotADev>

LINKEDIN

[/in/jasonmull](https://www.linkedin.com/in/jasonmull)

WEB

<https://jasonmull.com>



// REFERENCES

Where This Came From

01 **Model Context Protocol — spec**

<https://modelcontextprotocol.io/specification>

02 **Anthropic — Claude Code memory**

<https://docs.anthropic.com/en/docs/claude-code/memory>

03 **GitHub Spec Kit**

<https://github.github.com/spec-kit>

04 **Chainsaw — WithSecure Labs**

<https://github.com/WithSecureLabs/chainsaw>

05 **Sigma rules — SigmaHQ**

<https://github.com/SigmaHQ/sigma>

06 **ChainsawMCP — the server from this talk**

<https://github.com/jasonmull/ChainsawMCP>

07 **LM Studio**

<https://lmstudio.ai>

08 **Ollama**

<https://ollama.com>

09 **llama.cpp**

<https://github.com/ggml-org/llama.cpp>

10 **Foundation-Sec — Cisco Foundation AI**

<https://huggingface.co/fdtn-ai>

11 **llmfit — estimate before you download**

<https://llmfit.org>

12 **AIPerf — NVIDIA**

<https://github.com/ai-dynamo/aiperf>

13 **safetensors — model file format**

<https://huggingface.co/docs/safetensors>

14 **Ollama GGUF parser bug**

<https://nvd.nist.gov/vuln/detail/CVE-2026-7482>

15 **Claude Code — plugins & security review**

<https://docs.anthropic.com/en/docs/claude-code>

16 **GitHub code scanning — CodeQL**

<https://docs.github.com/code-security/code-scanning>

17 **Package hallucinations — USENIX Sec '25**

<https://arxiv.org/abs/2406.10279>

18 **Slopsquatting in the wild — Aikido**

<https://www.aikido.dev/blog>

19 **Karpathy — ramble sessions and interviews**

<https://x.com/karpathy/status/2079610838143623371>

20 **Hunt & Thomas — The Pragmatic Programmer**

<https://pragprog.com/titles/tpp20>

21 **Pillar Security — poisoned GGUF chat templates**

<https://pillar.security/blog/llm-backdoors-at-the-inference-level>

22 **Nguyen — Common AI Model Formats (Hugging Face)**

<https://huggingface.co/blog/ngxson/common-ai-model-formats>

